Intelligent Security for Containers:

ML-Based Runtime and Orchestration Threat Classification Using the Jev Model

A Project Report Extending "Container Security in Multi-Tenant Cloud Environments" from Image Scanning to Runtime and Orchestration Layers

A design-stage project report building on the exploit taxonomy of Jarkas, Ko, Dong & Mahmud (2025), "A Container Security Survey: Exploits, Attacks, and Defenses," ACM Computing Surveys

Prepared by:

Altaf Raja – 23BCS011

Kamal Nayan Kumar – 23BDS026

Himanshu Kumar – 23BCS055

Indian Institute of Information Technology Dharwad

Course: Cloud Security and SRE

Faculty Guide: Malay Kumar

Date: 5 October 2026

Table of Contents

Abstract..... 3
Keywords..... 3
1. Introduction..... 4
2. Background and Related Work..... 5
3. The Jev Model..... 6
4. System Architecture..... 8
5. Data and State Design..... 10
6. Classification Design with Jev..... 11
7. Decision Policy and Response..... 14
8. Evaluation Methodology..... 14
9. Risks and Limitations..... 15
10. Future Work..... 16
11. Conclusion..... 16
12. References..... 17

Abstract

Container platforms share a single host kernel across many tenants, so a compromise at the runtime, host, or orchestration layer can spread across workloads. Our earlier report, a synthesis of Jarkas et al. (2025) and four complementary studies, found that misconfiguration, weak isolation, and untrusted base images dominate container risk, and identified a gap: machine-learning classification has so far been demonstrated on container-image metadata but not on runtime behaviour or orchestration-layer signals (Ugale & Potgantwar, 2025). This report describes the design of a system that addresses that gap.

The proposed system collects telemetry from four sources in a Kubernetes laboratory cluster (image scans, runtime system calls, Kubernetes audit logs, and network flows), converts each suspicious event into a structured text state together with a per-workload behavioural baseline, and submits that state to Jev, a System One decision model released by TypeSafe AI in September 2026. Jev answers a fixed set of typed questions in a single parallel pass: a Choice question that selects the attack type from a defined set of classes, a Score question that rates severity, and Noul questions that estimate the probability of specific evidence flags. A decision engine combines the typed answers with their confidence values to alert, quarantine, escalate to a human reviewer, or terminate a workload, and every decision is logged to support evaluation and threshold tuning.

The report explains the system architecture, the internal working of the Jev model, the state and question design, the decision policy, the planned evaluation against rule-based and classical machine-learning baselines, and the risks and limitations of the approach. At the time of writing the system is at the design stage. No experimental results are reported, and the numerical values used in worked examples and thresholds are illustrative or provisional.

Keywords

Container security; Kubernetes; runtime threat detection; machine learning; classification; Jev; System One; Falco; eBPF; multi-tenant cloud; site reliability engineering.

1. Introduction

1.1 Background

Containers package an application with its dependencies at the operating-system level, so many containers on one host share a single kernel instead of each carrying a guest operating system. This is the source of their efficiency, and also the reason their isolation boundary is weaker than that of a virtual machine. In multi-tenant cloud environments, where workloads from different tenants run on shared Kubernetes clusters, a weakness in the container runtime, the host kernel, or the orchestration layer can therefore affect many workloads at once.

Our earlier report, “Container Security in Multi-Tenant Cloud Environments”, synthesized the taxonomy of Jarkas et al. (2025), which classifies more than 200 container-related CVEs into 47 exploit types across 11 attack vectors and ten mitigation-strategy categories spanning five architectural layers, and compared it with four complementary studies. Two findings from that report drive this project. First, independent research methods agree that misconfiguration, weak isolation, and untrusted base images dominate real-world container risk. Second, the report identified, as a future research direction, the extension of machine-learning-based risk classification beyond image-scan metadata to runtime behaviour and orchestration-layer signals, building on Ugale and Potgantwar (2025).

Documented incidents illustrate why those layers matter. In 2018 attackers compromised Tesla's Kubernetes console to run cryptocurrency-mining software on the company's cloud resources, and in 2019 attackers breached Docker Hub and exposed the credentials of roughly 190,000 accounts (Wong et al., 2023). The Tesla case in particular was an orchestration-level compromise that showed up as abnormal runtime resource use, which is the kind of behaviour a static image scan cannot observe.

1.2 Problem Statement

Most automation in container security, including the machine-learning example in the literature we reviewed, operates on static artifacts such as images. The attacks that cause the most damage, including container escape, privilege abuse, and resource hijacking, reveal themselves at runtime and in orchestration activity. The signals there are heterogeneous (system calls, API audit records, network flows) and depend heavily on context: a shell session in a debugging pod is routine, while the same session in a payment service is a warning sign.

Rule-based runtime tools detect known patterns precisely but must be written and maintained by hand and cope poorly with context. Classical machine-learning detectors need numeric feature engineering and large labelled datasets. The question this project asks is whether a fast, typed decision model can classify runtime and orchestration events when it is given the event, the relevant facts, and the workload's normal behaviour as text, and what pipeline is needed around such a model for it to be useful in a security operations setting.

1.3 Motivation

The motivation is direct. Our earlier report listed the extension of machine-learning classification from image scanning to runtime and orchestration as an open problem, and the course framing of Cloud Security and Site Reliability Engineering treats security as part of reliability rather than a separate step. A system that observes the running cluster, classifies suspicious behaviour quickly, and either acts or escalates fits that framing: it shortens detection time, defines a response, and produces an audit trail that supports continuous improvement.

1.4 Project Objectives

- To design an end-to-end architecture that collects image, runtime, orchestration, and network telemetry from a multi-tenant Kubernetes laboratory cluster and turns it into classifiable events.
- To define how a container event is represented as a structured text state, together with a per-workload behavioural baseline, so that context can inform classification.
- To use the Jev model as the classification step, with a Choice question for attack type, a Score question for severity, and Noul questions for evidence flags.
- To define a decision policy that uses the typed answers and their confidence values to alert, quarantine, escalate to a human, or terminate a workload.
- To plan an evaluation that compares Jev with a rule-based baseline and a classical machine-learning baseline on accuracy, latency, and cost.
- To identify the risks and limitations of placing a hosted decision model in a security pipeline, together with a mitigation for each.

1.5 Scope and Assumptions

This is a design-stage report. The pipeline has not yet been fully implemented, access to the hosted Jev API has not yet been obtained, and no experimental results are reported. All numerical values shown in worked examples, figures, and thresholds are illustrative or provisional and are labelled as such.

The work is limited to Linux containers orchestrated by Kubernetes in an isolated laboratory cluster controlled by the team. The initial classification targets three attack classes (container escape, privilege abuse, and cryptomining) plus benign and unknown outcomes. Attack scenarios are executed only inside that laboratory environment, and this report does not provide step-by-step exploitation instructions. It does not cover vendor-by-vendor product comparison, regulatory compliance analysis, or production deployment.

1.6 Significance

Academically, the project tests an under-explored direction identified in our earlier synthesis: whether machine-learning classification can be extended from image metadata to runtime and orchestration signals. Practically, it shows how a typed-decision model can be embedded in an SRE-style pipeline with explicit confidence gating, human review, and an improvement loop. Pedagogically, it connects the literature-based work of the earlier report to a concrete, testable system design.

2. Background and Related Work

2.1 The Container Risk Landscape

The taxonomy of Jarkas et al. (2025) organizes container exploits across five architectural layers, from orchestration through application, engine and runtime, and host, down to hardware. Its most granular subdivision is at the host layer, which accounts for 20 of the 47 exploit types, because shared-kernel mechanisms such as namespaces, system calls, eBPF, and kernel modules form a security boundary common to every container on the host. He et al. (2023) show that eBPF in particular can open cross-container attack paths in cloud environments, which reinforces the importance of the host layer.

The other sources in our earlier report broaden the picture. A systematic mapping study of 129 primary studies (Sroor et al., 2025a) and an interview study of 35 practitioners in eight countries (Sroor et al., 2025b) both report misconfiguration and image trust as dominant concerns, and Haque and Babar (2022) show that base-image vulnerabilities can remain exploitable even when their CVE

severity scores are low. Taken together, these results suggest that useful automation should reason about context and configuration, not only about severity scores.

2.2 From Image Scanning to Runtime Intelligence

Ugale and Potgantwar (2025) apply machine learning to classify the risk of Docker images from vulnerability metrics, and our earlier report summarized their reported classification accuracy figures of 90.91% and 100%. Their work establishes that learned classifiers can automate part of the mitigation workflow, but its input is image-scan metadata. It does not address what a container does once it is running, nor what happens in the orchestration layer. Extending the idea to those layers is the gap this project targets.

2.3 Common Runtime Detection Approaches

Two approaches are common in practice. The first is rule-based detection, in which rules match system calls or audit events against known suspicious patterns. Rules are transparent and precise for known behaviour, but they require continuous manual authoring and have no natural way to express that the same action is normal for one workload and suspicious for another. The second is classical machine-learning anomaly detection, in which events are converted to numeric feature vectors and a model such as a random forest or an isolation forest scores them. This handles variation better, but it needs feature engineering, labelled or representative training data, and retraining as workloads change.

A typed decision model offers a third option. The event and its context are written as text, and the model is asked fixed questions whose answers are constrained to a known set. The model supplies the contextual judgment, while thresholds, policy, and actions remain in ordinary code.

2.4 Positioning of This Project

This project does not claim to replace rule-based or classical approaches. It treats them as baselines and as components: cheap rules act as a pre-filter in front of the model, and classical classifiers serve as a comparison in the evaluation. The contribution is the design of a pipeline that connects four telemetry sources to a typed classification step and a policy-driven response, and the plan to measure that design honestly against alternatives.

3. The Jev Model

3.1 What Jev Is

Jev is the first public model in a family that its developer, TypeSafe AI, calls System One. It was released in early access in mid-September 2026 (WPS, 2026). Instead of generating text, it evaluates a supplied state and returns typed answers, with probabilities, that application code can branch on directly. The name System One refers to fast, intuitive judgment, and TypeSafe reports that the model was trained with a method it calls Reinforcement Learning for Calibrated Decisions (WPS, 2026).

Because Jev is new, no peer-reviewed evaluation of it exists yet. The description in this section relies on vendor-described specifications and secondary technical write-ups (CometAPI, 2026; Composio, 2026; Opentweet, 2026; WPS, 2026), and performance figures such as response time are vendor-reported and not independently verified. Verifying them is part of the planned evaluation.

3.2 Anatomy of a Request

A request to Jev has three parts. The **state** is the context to be judged, supplied as text or JSON. The **questions** are defined in advance, and each has a type. The **model identifier** selects either a fixed version or a moving alias. Jev reads the state once and evaluates each question against it. TypeSafe states that the questions in a request are evaluated independently, so no answer becomes input to

another, and that adding independent questions has little effect on response time, with end-to-end responses of roughly 70 to 500 milliseconds (CometAPI, 2026).

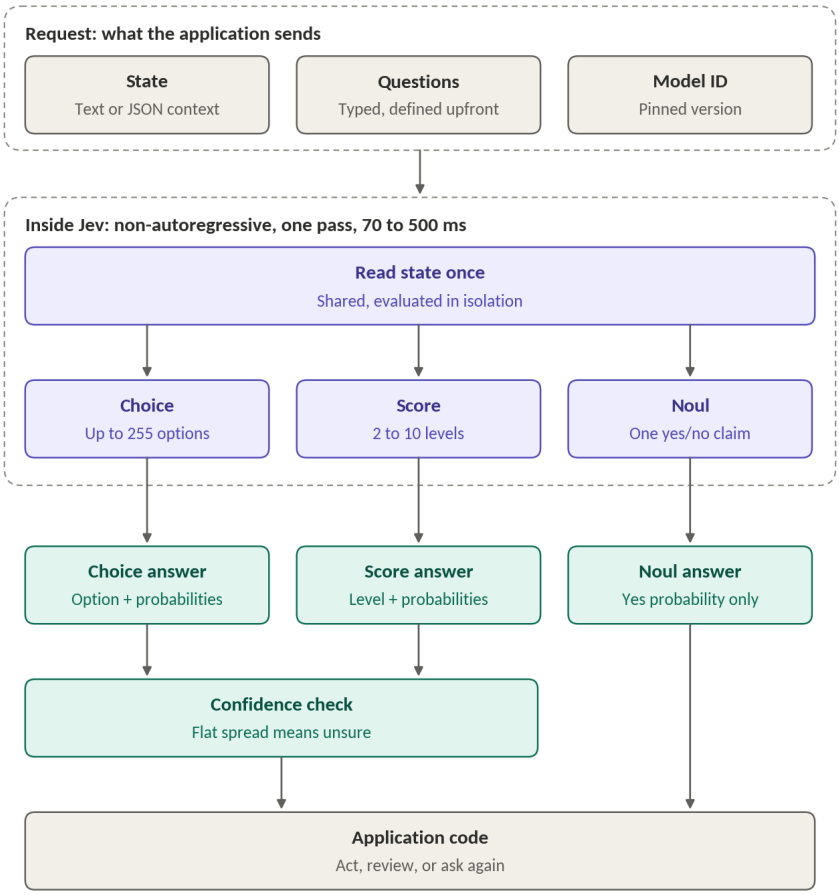


Figure 1. Internal working of the Jev model: request, parallel typed questions, typed answers, and the application's confidence check.

3.3 Question Types

Every question is one of three types, summarized in Table 1. All three are sent to the same endpoint in a single request body containing the state, the model, and the questions (Opentweet, 2026).

Table 1. Jev question types.

Type	What the application defines	What Jev returns	Confidence field
Choice	A fixed set of options (up to 255), each with a description	The chosen option and a probability for every option	Yes
Score	An ordered rubric of levels (2 to 10), each with a description	A score that may fall between levels, a legend, level probabilities	Yes
Noul	One yes/no claim	A single probability, from 0 to 1, that the claim is true	No

3.4 Confidence and Probabilities

For Choice and Score, the confidence value summarizes how strongly the probability distribution favours a single outcome. A sharply peaked distribution indicates a clear answer, while a flat distribution means that the state, the question, or the option descriptions need another look. A Noul

answer has no separate confidence, because the probability is itself the answer: values near 0 or 1 are strong, and values near 0.5 mean the model is unsure. A two-level Score can be used when a yes/no decision needs a confidence value (Opentweet, 2026). These properties matter for this project because they allow the decision engine to refuse automatic action when the model is uncertain.

3.5 Versioning and Access

TypeSafe provides both fixed model identifiers and moving aliases. At the time of writing the stable version is identified as jev-1.13.0, and the alias jev-latest points to the latest stable release (CometAPI, 2026). The hosted service is reached through an HTTP endpoint, and Python and JavaScript client libraries are available (Composio, 2026). The service is in early access, and the provider notes that rate limits may be adjusted dynamically (WPS, 2026).

Access to the hosted API has not yet been obtained for this project. The design therefore treats the classification step as a replaceable component behind a narrow interface (state and questions in, typed answers out), so that the same pipeline can later be pointed at the hosted model, a fine-tuned open model that produces the same kind of typed output, or a baseline classifier for comparison.

3.6 Suitability and Limitations

Jev fits this project because the task is a bounded judgment over a short text description, repeated at high volume, where the answer must be one of a known set. It is not suited to deterministic computation such as counting events, comparing timestamps, or checking whether a specific path appears in a mount list, and those checks belong in ordinary code. The model also does not explain its reasoning. For a security pipeline this means the audit trail must record the exact state sent, the typed answers, the probabilities, and the model version, because no free-text rationale will be available afterwards.

4. System Architecture

4.1 Overview

Figure 2 shows the complete data flow. Telemetry from the laboratory cluster passes through a normalizer that builds a text state, a pre-filter that removes clearly normal events, the Jev classification step, and a decision engine whose output is either a human review or an automated response. Every decision is written to an audit log, and the logged events feed an improvement loop in which analysts label events, the team evaluates the results, and thresholds, option descriptions, or the model are tuned.

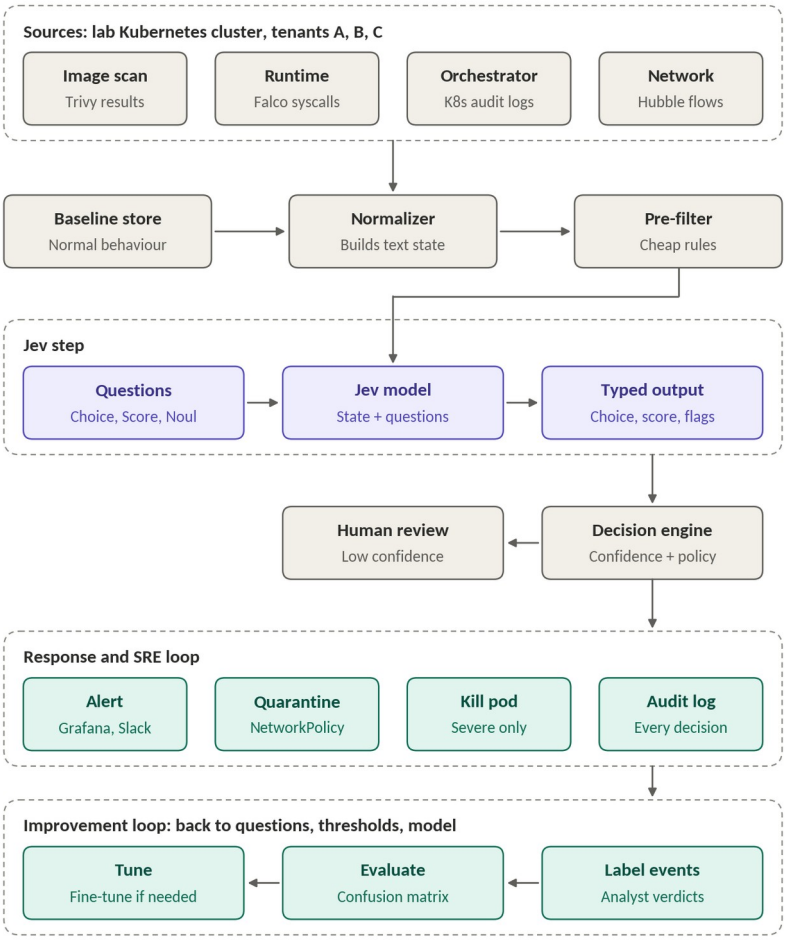


Figure 2. End-to-end data flow of the proposed intelligent container security system.

4.2 Design Principles

- **Context before classification.** Each event is paired with the workload's normal behaviour, because the same action can be benign or malicious depending on the workload.
- **Cheap before expensive.** Simple rules and thresholds remove obviously normal events so that the model sees only suspicious ones.
- **Typed outputs.** Every question has a fixed answer type, so the downstream code never parses free text.
- **Deterministic where possible.** Facts that can be computed exactly, such as whether a sensitive host path is mounted, are extracted by code and given to the model as facts.
- **Humans in the loop.** Low-confidence and unknown cases go to a reviewer, and the most disruptive actions are disabled until they have been validated.
- **Everything logged.** Each decision records the state, answers, probabilities, model version, and action, so it can be audited and reused as evaluation data.

4.3 Components

Table 2. Components of the proposed pipeline.

Component	Role	Planned tooling	Output
Sources	Collect raw signals from four layers	Trivy, Falco, Kubernetes audit logs, Hubble	Raw events
Baseline store	Record normal behaviour per workload	Observation period in the lab cluster	Baseline summary
Normalizer	Build the text state and extract deterministic facts in code	Python service	Structured text state
Pre-filter	Drop clearly normal events	Rules and rate thresholds	Suspicious events
Jev step	Classify attack type, severity, and evidence	Hosted Jev API (access pending)	Typed answers
Decision engine	Apply confidence gate and policy	Application code	Action and reason
Response	Alert, quarantine, or terminate	Grafana, Slack, NetworkPolicy, pod deletion	Actions
Audit log	Record every decision	Log store	Labelled evaluation data

The four sources correspond to the layers discussed in Section 2.1. Trivy results extend the image-scanning work of Ugale and Potgantwar (2025) into the pipeline as one input among several. Falco observes runtime system calls at the engine and host layers, Kubernetes audit logs expose orchestration-layer actions such as role changes and executions into pods, and Hubble, the observability component of Cilium, provides network flows between workloads.

5. Data and State Design

5.1 Laboratory Environment

The system will be built on a local Kubernetes cluster created with a lightweight tool such as kind or minikube. Three namespaces emulate tenants A, B, and C, matching the multi-tenant setting of the earlier report, and each runs a small set of representative workloads such as a web front end, an API service, a database, and a payment service. A baseline observation period records normal process, mount, and network behaviour for each workload before any attack scenario is run.

5.2 Attack Scenarios

Scenarios are executed only in the isolated laboratory cluster and are described here at the level of intent, not procedure. They are chosen to match the classes in Section 6 and the taxonomy of Jarkas et al. (2025). Container escape scenarios attempt to reach host resources from inside a container, for example through an exposed runtime socket, a sensitive host path, or privileged mode. Privilege abuse scenarios misuse cluster permissions, for example through an over-permissive role or a service-account token. Cryptomining scenarios run an unexpected compute-heavy process that opens outbound connections. Deliberately vulnerable training environments for Kubernetes can supply additional scenarios, and benign activity such as deployments, restarts, and routine administration is recorded alongside the attacks so that the classifier is also tested on normal behaviour.

5.3 Event-to-State Template

Every event that passes the pre-filter is rendered in a fixed template, because a consistent format makes the model's inputs comparable across events. Table 3 lists the fields. Deterministic facts, such as whether a host runtime socket is mounted, are computed by the normalizer and written into the state as facts rather than left for the model to infer.

Table 3. Fields of the text state.

Field	Source	Why it matters
Namespace, pod, image	Kubernetes metadata, Trivy	Identifies the tenant and workload, and carries image findings
Process and parent process	Falco	Shows what ran and what started it
System call or action	Falco, audit log	Describes the observed behaviour
Mounts and privileges	Pod specification	Exposes host access and privileged settings
Service account and API action	Kubernetes audit log	Shows orchestration-layer permission use
Network destination	Hubble	Shows unexpected outbound or lateral connections
Resource use	Metrics	Supports detection of sustained compute abuse
Baseline summary	Baseline store	States what is normal for this workload

5.4 Labelling and Dataset

Each event is labelled with its class using the known ground truth of the scenario that produced it, and benign events are labelled as such. The dataset is divided into a tuning set, used to refine descriptions, thresholds, and any fine-tuning, and a held-out test set that is fixed before tuning begins, so that reported results are not inflated by tuning on the test data. Publicly available system-call datasets for intrusion detection may be used to supplement the laboratory data, with the limitation that they were not produced in this cluster.

6. Classification Design with Jev

6.1 Attack Type (Choice)

The central question asks which attack pattern best matches the event. The options and their descriptions are given to Jev with every request, and the description of each option states the observable signals that distinguish it, because the model can use those descriptions when choosing. Table 4 lists the initial options and their relation to the taxonomy. The class set is configurable, and the benign and unknown options are always present so that the model is never forced to label a normal event as an attack.

Table 4. Initial options for the attack-type Choice question.

Option	Description given to Jev (summary)	Link to taxonomy and incidents
container_escape	Access to host resources or the runtime: exposed runtime socket, host path, privileged mode, host namespaces	Host and runtime layers: shared-kernel mechanisms
privilege_abuse	Misuse of cluster permissions: role changes, service-account token use, secret reads, execution into other pods	Orchestration layer: misconfiguration
cryptomining	Sustained high CPU, mining-pool traffic, unknown binaries running from temporary paths	Resource hijacking, as in the 2018 Tesla incident
benign	Normal behaviour for this workload, including routine administration and deployment	Not applicable
unknown	Evidence is insufficient or ambiguous	Routes the event to human review

6.2 Severity (Score)

Severity is asked as a Score question over an ordered rubric of five levels: informational, low, medium, high, and critical, each with a short description. Jev returns a score that can fall between levels, for example 4.7, together with the probability of each level and a confidence value. The score is used for ranking and thresholding, not as a precise measurement. Informational means expected activity, low means unusual but plausibly benign, medium means a suspicious policy violation that needs review, high means likely malicious activity or exposure of a sensitive resource, and critical means indicators of active compromise that call for immediate containment.

6.3 Evidence Flags (Noul)

Alongside the Choice and Score questions, a small number of Noul questions ask for judgments that cannot be computed exactly. They give the decision engine independent evidence with which to cross-check the Choice answer. Table 5 lists the initial flags.

Table 5. Initial Noul evidence flags.

Flag	Claim posed to Jev	Use in decisions
baseline_deviation	This activity is inconsistent with the workload's normal behaviour.	Supports escalation of ambiguous events
shell_unexpected	An interactive shell session is unexpected for this workload.	Corroborates escape and abuse classes
resource_exposure	The event gives the workload access to host or cluster resources it should not hold.	Corroborates container_escape
resource_abuse	Resource or network behaviour is consistent with unauthorized compute use.	Corroborates cryptomining

6.4 Worked Example

Figure 3 follows one event through the Jev step. A shell process started by an nginx container in the payment workload of tenant B, with a host runtime socket mounted, is rendered as a text state together with the baseline note that nginx pods normally do not start a shell. The three questions are evaluated in one parallel pass. In this illustration Jev returns container_escape with high probability, a severity score of 4.7, and a high probability that the shell is unexpected, and the application code

applies its policy and quarantines the workload. The values in the figure are illustrative and are not measured results.

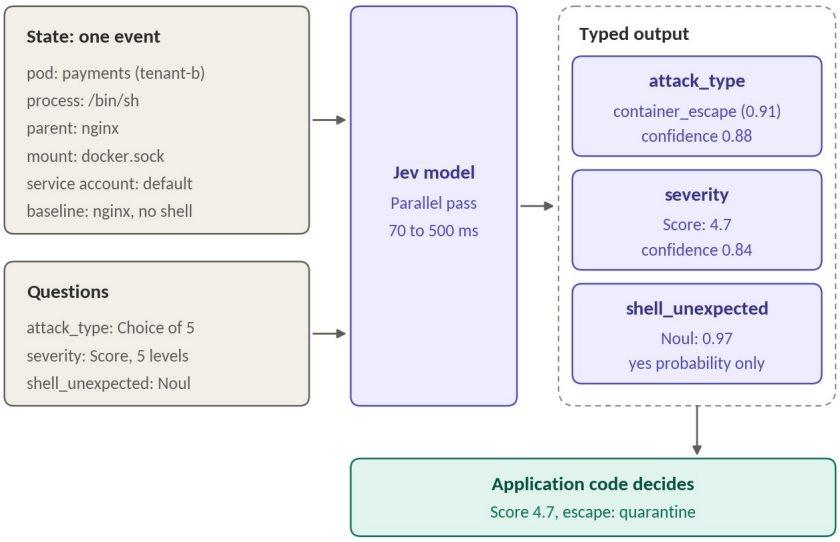


Figure 3. Worked example of one event passing through the Jev step (illustrative values).

Listing 1 shows the indicative shape of the request for this event. Exact field names follow the provider's documentation and may differ.

```
{
  "model": "jev-1.13.0",
  "state": {
    "event": "ns=tenant-b pod=payments proc=/bin/sh parent=nginx",
    "facts": "host runtime socket mounted; service account=default",
    "baseline": "nginx pods normally never start a shell"
  },
  "questions": {
    "attack_type": {
      "type": "choice",
      "options": {
        "container_escape": "access to host resources or the runtime",
        "privilege_abuse": "misuse of cluster permissions or tokens",
        "cryptomining": "sustained compute use, mining-pool traffic",
        "benign": "normal behaviour for this workload",
        "unknown": "evidence is insufficient or ambiguous"
      }
    },
    "severity": {
      "type": "score",
      "levels": ["informational", "low", "medium", "high", "critical"]
    },
    "shell_unexpected": { "type": "noul" }
  }
}
```

Listing 1. Indicative request for the worked example.

6.5 Helping the Model Choose Correctly

Giving the model several options does not by itself make it choose well. The design uses six measures, and several of them are tested in the ablation study of Section 8.

- **Discriminating descriptions.** Each option states the observable signals that separate it from the others, not just a definition.
- **Evidence and baseline in the state.** The state contains the facts that decide between classes, and the workload's normal behaviour, so that context is available.
- **Exclusive options and a fallback.** Overlapping options are given explicit boundaries, and benign and unknown are always offered so that the model is never forced into an attack class.
- **Independent evidence flags.** Noul questions provide a second signal, and a disagreement between the Choice answer and the flags can trigger review.
- **A confidence gate.** If the top probability or the confidence is below a threshold, the decision engine does not act automatically. The threshold is calibrated on the tuning set.
- **Domain adaptation if needed.** If accuracy is insufficient after the descriptions and state are improved, a fine-tuned model that produces the same typed output can be trained on the labelled laboratory data.

7. Decision Policy and Response

The decision engine is ordinary code, not part of the model, so its behaviour is predictable and auditable. It reads the typed answers and confidence values and applies the rules in Table 6. The thresholds are provisional starting values and are calibrated on the tuning set. In the first phase, termination of workloads is disabled and quarantine is applied only after review, and automation is enabled stage by stage once the evaluation shows acceptable false-positive rates.

Table 6. Initial decision rules (provisional thresholds).

Condition	Action
Attack type is benign and confidence is at or above the threshold	Log only
Attack type is unknown, or confidence is below the threshold (initially 0.6)	Route to human review
Attack class with severity below 3.0	Raise a low-priority alert
Attack class with severity from 3.0 to below 4.0	Raise an alert and request review
Attack class with severity of 4.0 or above and confidence at or above the threshold	Quarantine the workload with a network policy and alert
Severity of 4.5 or above with escape evidence, once automation is validated	Terminate the pod and alert

This design reflects the SRE view that security is part of reliability. A false positive that terminates a healthy payment service is an availability incident, so quarantine is preferred to termination because it can be reversed, uncertain cases go to people, and each action is recorded with its reason, which supports post-incident review and rollback.

8. Evaluation Methodology

8.1 Baselines

Jev will be compared with two baselines on the same held-out events. The first maps the alerts of a default rule set for runtime detection to the same classes. The second is a classical classifier, such as a

random forest, trained on engineered features from system-call counts and audit-log fields. If a fine-tuned model producing the same typed output is built, it is added as a further comparison.

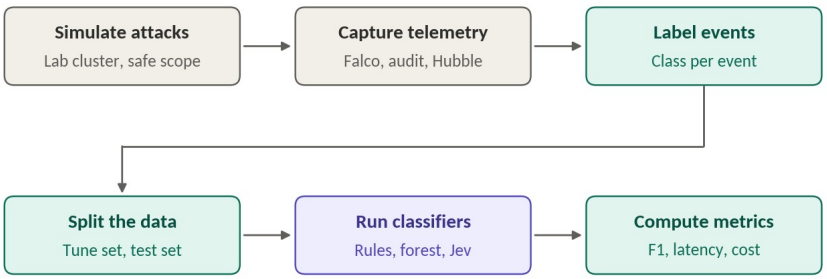


Figure 4. Planned evaluation pipeline.

8.2 Metrics

- Per-class precision, recall, and F1, the macro-averaged F1, and the confusion matrix.
- The false-positive rate on benign events, because it determines the operational cost of automation.
- Calibration: whether higher confidence corresponds to higher accuracy.
- Severity quality: rank agreement between the Score output and analyst-assigned severity.
- End-to-end latency, reported at the median and the 95th percentile, and cost per thousand events.

8.3 Protocol and Ablations

Each classifier is run on the held-out set after tuning is complete. Ablations isolate the contribution of each design decision: with and without the baseline in the state, with and without option descriptions, with and without the pre-filter, and across a sweep of confidence thresholds. The comparison with the rule-based baseline tests the claim that context helps, and the comparison with the classical classifier tests whether the typed-decision approach reduces the need for feature engineering.

8.4 Reproducibility

Runs use a fixed model version rather than a moving alias, because an alias can point to a newer release and change behaviour. The state template, option descriptions, thresholds, and the data split are version-controlled, every request and response is logged with the model version, and the test set is not touched during tuning.

9. Risks and Limitations

Table 7 summarizes the main risks of the approach and the planned mitigation for each. The most important is that an attacker may control some of the text that reaches the model, such as process arguments or image names, and could try to manipulate the classification. The pipeline therefore treats such fields as untrusted data.

Table 7. Risks and mitigations.

Risk	Consequence	Mitigation
Attacker-controlled text in the state	The classifier may be steered toward benign	Sanitize and truncate fields, delimit untrusted text, and let deterministic checks override for critical facts

Risk	Consequence	Mitigation
Telemetry sent to a hosted API	Exposure of identifiers or secrets	Redact secrets, minimize fields, and keep a self-hosted option
Early-access dependency	Rate limits or outages	Fall back to the rule-based baseline, and pin the model version
Latency and cost at scale	Delays and expense	Pre-filter, and send only suspicious events
False positives	Availability incidents	Human review, reversible quarantine, and staged automation
No explanation from the model	Weak audit trail	Log state, answers, probabilities, and version, and use Noul flags as evidence
Synthetic laboratory data	Results may not generalize	State this limitation, include diverse benign activity, and test on public data
Version change	Behaviour drift	Pin versions and re-run the evaluation before any upgrade

Beyond these risks, the main limitations of this report are that it describes a design and not a finished system, that the vendor-reported properties of Jev are not yet independently verified, that the initial class set is narrow, and that a small laboratory cluster cannot represent all production conditions.

10. Future Work

- Extend the class set to lateral movement, secret exfiltration, and supply-chain events, and use a two-stage Choice if the number of options approaches the limit.
- Include the image-scan result as part of the state, so that image risk and runtime behaviour are classified together.
- Combine severity with operational context such as exposure, usage, and business impact into a unified risk metric, as recommended by the open problems in our earlier work.
- Validate exploitability beyond base images by testing at runtime and orchestration layers, as suggested by Haque and Babar (2022).
- Study human and organizational factors by involving practitioners in labelling and reviewing decisions, following the direction of Sroor et al. (2025b).
- Evaluate fine-tuned open models as a self-hosted alternative to the hosted service.

11. Conclusion

Containers gain their efficiency by sharing a kernel, and the same choice makes runtime and orchestration compromise a cluster-wide concern. Our earlier report found that machine-learning classification had been shown for container images but not for runtime behaviour or orchestration signals. This report proposes a concrete design for closing that gap: four telemetry sources feed a normalizer and pre-filter, the Jev model classifies each suspicious event through typed Choice, Score, and Noul questions in a single parallel pass, and a policy-driven decision engine alerts, quarantines, escalates, or terminates, with every decision logged for evaluation and improvement.

The approach is promising because it keeps judgment in the model and control in code, exposes confidence for safe gating, and fits the SRE view of security as part of reliability. Its value, however, remains to be demonstrated. The next steps are to obtain access to the model, implement the pipeline in the laboratory cluster, generate and label the dataset, and run the comparison against rule-based and classical baselines described in Section 8, reporting the results and limitations honestly.

12. References

References are presented in APA 7th edition style, in alphabetical order by first author's surname.

CometAPI. (2026). Jev by TypeSafe AI: Model specifications and how it works. https://www.cometapi.com/en/models/typesafe-ai/jev

Composio. (2026). TypeSafe Jev guide: How it works and 6 best practical use cases. https://composio.dev/content/typesafe-jev-guide-and-best-practical-usecases

Haque, M. U., & Babar, M. A. (2022). Well begun is half done: An empirical study of exploitability and impact of base-image vulnerabilities. In 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) (pp. 1066–1077). IEEE. (Preprint: arXiv:2112.12597)

He, Y., Guo, R., Xing, Y., Che, X., Sun, K., Liu, Z., Xu, K., & Li, Q. (2023). Cross container attacks: The bewildered eBPF on clouds. In Proceedings of the 32nd USENIX Security Symposium (pp. 5971–5988). USENIX Association.

Jarkas, O., Ko, R., Dong, N., & Mahmud, R. (2025). A container security survey: Exploits, attacks, and defenses. ACM Computing Surveys, 57(7), Article 170, 1–36. https://doi.org/10.1145/3715001

Opentweet. (2026). Jev Choice vs Score vs Noul. https://opentweet.io/jev/choice-score-noul

Sroor, M., Das, T., Mohanani, R., & Mikkonen, T. (2025a). A systematic mapping study on risks and vulnerabilities in software containers (arXiv:2512.11940). arXiv. https://arxiv.org/abs/2512.11940

Sroor, M., Mohanani, R., Colomo-Palacios, R., Dasanayake, S., & Mikkonen, T. (2025b). Managing security issues in software containers: From practitioners' perspective (arXiv:2504.07707). arXiv. https://arxiv.org/abs/2504.07707

Ugale, S., & Potgantwar, A. (2025). Risk classification of Docker container images using machine learning and vulnerability metrics. Engineering, Technology & Applied Science Research, 15(6), 30310–30316. https://doi.org/10.48084/etasr.12627

Wong, A. Y., Chekole, E. G., Ochoa, M., & Zhou, J. (2023). On the security of containers: Threat modeling, attack analysis, and mitigation strategies. Computers & Security, 128, Article 103140. https://doi.org/10.1016/j.cose.2023.103140

WPS. (2026). What is Jev? TypeSafe's System One model and its best use cases. https://www.wps.com/blog/what-is-jev-typesafe-s-system-one-model-and-its-best-use-cases/